

Graph

terça-feira, 19 de agosto de 2025 20:28

Perfect 🍷 since you already have a **static approach** and your entry data is a container of ClassST objects (with info like *class name* and *base classes*), let's walk step by step on how to build the **graph of class inheritance**.

1. Define what you already have (ClassST)

Let's assume your ClassST looks like this (simplified):

```
struct ClassST {
    std::string name;
    std::vector<std::string> baseClasses; // names of parent classes
};

And you have a container (maybe from your parsing stage):
std::vector<ClassST> classes;
Example:
classes = {
    {"A", {}},
    {"B", {"A"}},
    {"C", {"A"}},
    {"D", {"B", "C"}}
};
```

Let's do a adjacent + reverse edges

2. Choose a graph representation

We want a **directed graph** (edge from derived → base). Two good representations:

- **Adjacency list** (most common, easy to traverse):
std::unordered_map<std::string, std::vector<std::string>> graph;
- **Adjacency list + reverse edges** (if you also want "who inherits from me" queries):
std::unordered_map<std::string, std::vector<std::string>> parents; // derived → base
std::unordered_map<std::string, std::vector<std::string>> children; // base → derived

3. Build the graph

We iterate over your container and insert edges.

```
void buildGraph(
    const std::vector<ClassST>& classes,
    std::unordered_map<std::string, std::vector<std::string>>& graph
){
    for (const auto& cls : classes) {
        // Ensure the node exists, even if it has no bases
        if (!graph.count(cls.name)) {
            graph[cls.name] = {};
        }
        // Add edges from cls → base
        for (const auto& base : cls.baseClasses) {
            graph[cls.name].push_back(base);
        }
    }
    // Ensure base is also in the graph (in case it has no definition in container)
    if (!graph.count(base)) {
        graph[base] = {};
    }
}
```

After calling this with the example above, graph looks like:

```
A: []
B: [A]
C: [A]
D: [B, C]
```

4. Optional: Build reverse graph

If you want to also know *who derives from whom*:

```
void buildReverseGraph(
    const std::vector<ClassST>& classes,
    std::unordered_map<std::string, std::vector<std::string>>& children
){
    for (const auto& cls : classes) {
        for (const auto& base : cls.baseClasses) {
            children[base].push_back(cls.name);
        }
    }
}
```

Now children looks like:

```
A: [B, C]
B: [D]
C: [D]
D: []
```

5. Usage examples

- **Find all ancestors of D:** DFS on graph.
- **Find all descendants of A:** DFS on children.
- **Detect multiple inheritance cycles:** run a cycle detection algorithm on graph.

- ✔ With this setup:
- Entry = std::vector<ClassST>
 - Output = adjacency list(s)
 - Easy to traverse, visualize, or export (e.g., to Graphviz / Excalidraw later)

Do you want me to **write the full code (ClassST, buildGraph, buildReverseGraph, plus a DFS traversal example)** so you can plug it into your project directly?

Why to store strings and not tokens

Awesome — let's unpack that design in detail and fix a couple of important pitfalls in your new member layout.

You said your goal is a graph with **adjacency** (derived → bases) and **reverse** (base → derived). That shape is perfect. The only sticky point is **what you use as node keys**.

The single most important decision: node identity

Do not key your maps by std::shared_ptr<Token>

- unordered_map<std::shared_ptr<Token>, ...> compares **pointer addresses** by default, not token contents.
- Two different IDENTIFIER tokens with lexeme "Foo" will be treated as **different keys** if they are distinct allocations — that will fragment your graph.
- Tokens are also **transient** parser artifacts; your graph should outlive tokens and be independent of scanning/parsing lifetimes.

Use a canonical name as the key (for now: std::string)

- Today you don't have namespaces, so the class **name alone** is a stable identity.
- Later, you can switch to a ClassId struct with {namespace, name} and a custom hasher without changing the rest of the graph code.

So your storage should be:

```
// derived → bases (parents)
std::unordered_map<std::string, std::vector<std::string>> m_parents;
// base → derived (children)
std::unordered_map<std::string, std::vector<std::string>> m_children;
// all known nodes
std::unordered_set<std::string> m_nodes;
size_t m_edgeCount = 0;

Your proposal:
```

```
using baseClassPair = std::pair<std::shared_ptr<Token>, std::shared_ptr<Token>>;
std::unordered_map<std::shared_ptr<Token>, baseClassPair> m_parents;
std::unordered_map<std::shared_ptr<Token>, std::vector<baseClassPair>> m_children;
std::unordered_set<std::shared_ptr<ClassST>> m_nodes;
```

Problems:

- The **shape** is wrong: m_parents should map one **derived** to **many bases** (vector), not to a single pair.
- Keys based on pointers = identity by address, not by class name = duplicates.
- m_nodes as shared_ptr<ClassST> also keys by address; two ClassST objects with the same name would appear as different nodes.

If you absolutely want token-based keys, you must (a) ensure **unique ownership** of every token instance (so address identity is stable), or (b) supply custom hash and equality that compare **lexemes**. Option (a) is fragile; option (b) turns token pointers into "string keys in disguise". Using std::string directly is simpler and safer.